

Course: Image Processing 31651

Assignment #23

Pixel to Pixel Operations (Part 3)

	ID (4 last digits)	Shorten Name	Photo of the student	
Student #1	1950	shienfeld		
Student #2	2210	pony		
Student #3	7939	akimov		

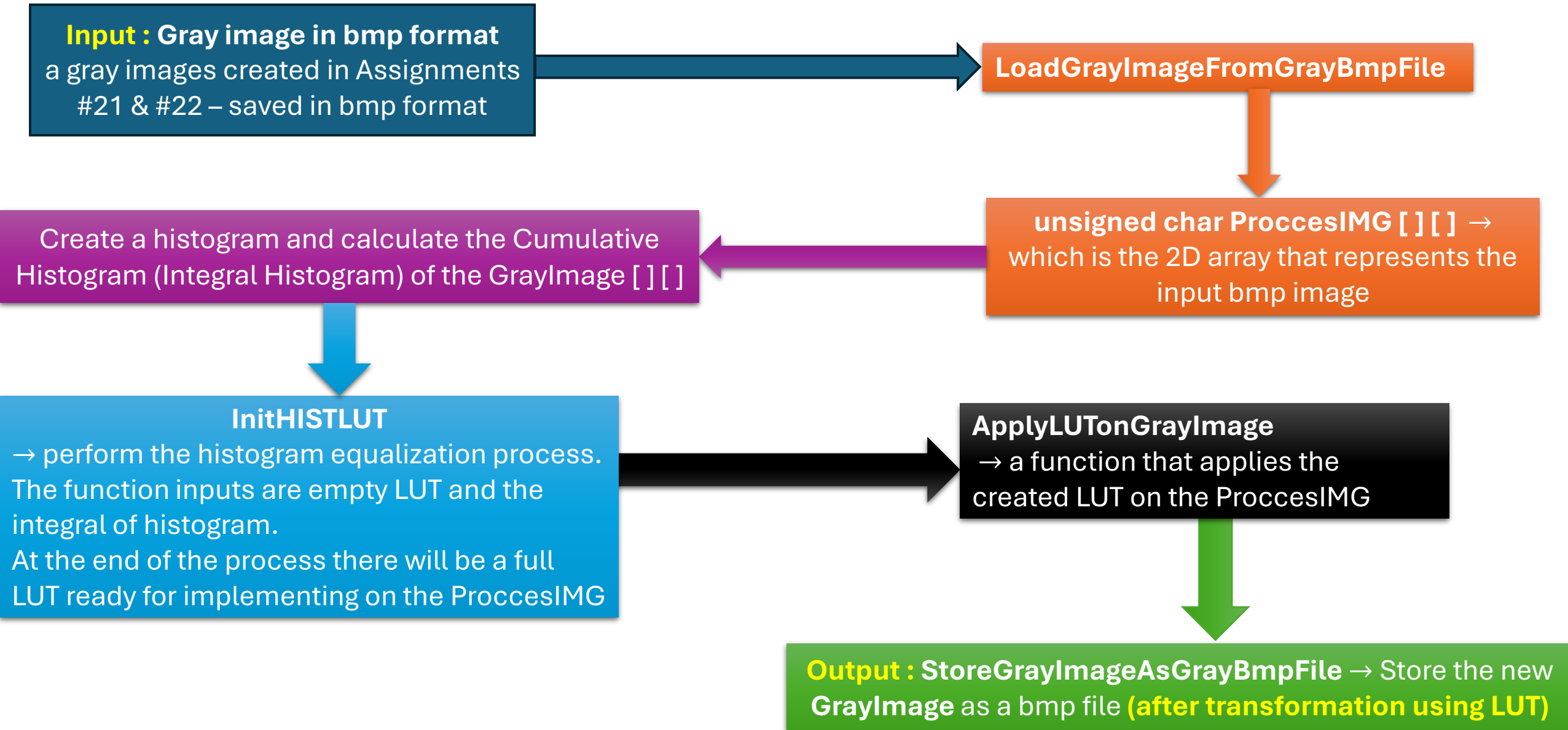
Assignment #23: Process set of Gray Images

In the file ImProcInPlainC.h defined numerical values of:
NUMBER_OF_ROWS and NUMBER_OF COLUMNS

#	Image Description
0	Use earlier prepared gray images
1	Apply algorithm Histogram Equalization to A20gray.bmp → A230.bmp Apply algorithm Histogram Equalization to A211.bmp → A231.bmp Apply algorithm Histogram Equalization to A212.bmp → A232.bmp Apply algorithm Histogram Equalization to A221.bmp → A233.bmp Apply algorithm Histogram Equalization to A222.bmp → A234.bmp The algorithm must use pointers to access the pixels Compare resulted images and explain results. Think how to create report in a fast and efficient way first, start to work only after thinking
	For report use YOUR OWN STYLE. From YOUR Power Point customer must understand what was done, how and what are the results (Lecturer may later ask to add slides) Done: 23.1 REMOVE ALL RED when the items are ready Important hint: Fill report pages while working. Do not complain “not enough time” in case you did not follow THIS rule.

23.0 - flow chart of the code – general look

3



23.1 – first important function – fill_hist_and_calc_int

```
15 // Function to fill histogram and calculate integral histogram
16 void fill_hist_and_calc_int(unsigned char* img, unsigned int hist[PIXEL_VALUES], unsigned int IntOfHist[PIXEL_VALUES], int rows, int cols) {
17     // Initialize histogram to zero
18     for (int i = 0; i < PIXEL_VALUES; i++) {
19         hist[i] = 0;
20     }
21
22     // Calculate histogram
23     for (int i = 0; i < rows * cols; i++) {
24         hist[*img + i] += 1;
25     }
26
27     // Calculate integral histogram
28     IntOfHist[0] = hist[0];
29     for (unsigned i = 1; i < PIXEL_VALUES; i++) {
30         IntOfHist[i] = IntOfHist[i - 1] + hist[i];
31     }
32 }
```

The histogram array is initialized to zero to ensure accurate counts of pixel values.

Iterate through each pixel in the image using pointer arithmetic to create the histogram of the image.

Compute the cumulative distribution of pixel intensities by summing the histogram values.

The Purpose of the `fill_hist_and_calc_int` function is To calculate the histogram and integral histogram of a grayscale image, which are essential for the histogram equalization.



For that assignment only the integral of the histogram is really needed. In 23.2 we will present the creation process of the LUT using the integral of histogram – that process is the histogram equalization

23.2 – second important function – **InitHISTLUT**

```
34 // Function to initialize LUT based on integral histogram
35 void InitHISTLUT(unsigned char* LUT, unsigned int* IntegralHist) {
36     // Scaling factor to map the integral histogram values to the range [0, 255]
37     double scaleFactor = (double)(myMAXCOLORS - 1) / (NUMBER_OF_ROWS * NUMBER_OF_COLUMNS);
38
39     // Iterate through each possible pixel value
40     for (int i = 0; i < myMAXCOLORS; i++) {
41         // Calculate the scaled value from the integral histogram
42         double scaledValue = scaleFactor * IntegralHist[i] ;
43
44         // Clamp the value to the range [0, 255]
45         if (scaledValue > 255) {
46             scaledValue = 255;
47         }
48         else if (scaledValue < 0) {
49             scaledValue = 0;
50         }
51
52         // Assign the scaled and clamped value to the LUT
53         LUT[i] =scaledValue;
54     }
55 }
```

The Purpose of **InitHISTLUT** function is perform the histogram equalization process. The function takes as inputs the (already filled) integral of histogram (created before) and an empty array as LUT - it Iterate through each possible pixel intensity value and calculate the corresponding new intensity value using the integral histogram and the normalization factor.

23.3 – third important function – **ApplyLUTOnGrayImage**

```
58 // Function to apply LUT on gray image
59 void ApplyLUTOnGrayImage(unsigned char* img, unsigned char* LUT, int rows, int cols) {
60     unsigned char* end = img + (rows * cols); // Pointer to the end of the image data
61     while (img < end) {
62         *img = LUT[*img];
63         img++;
64     }
65 }
```

This function applies the previously created LUT to the image. The purpose is to transform the original pixel values to new values based on the LUT for histogram equalization.

Pay attention that we use pointers throughout the program.

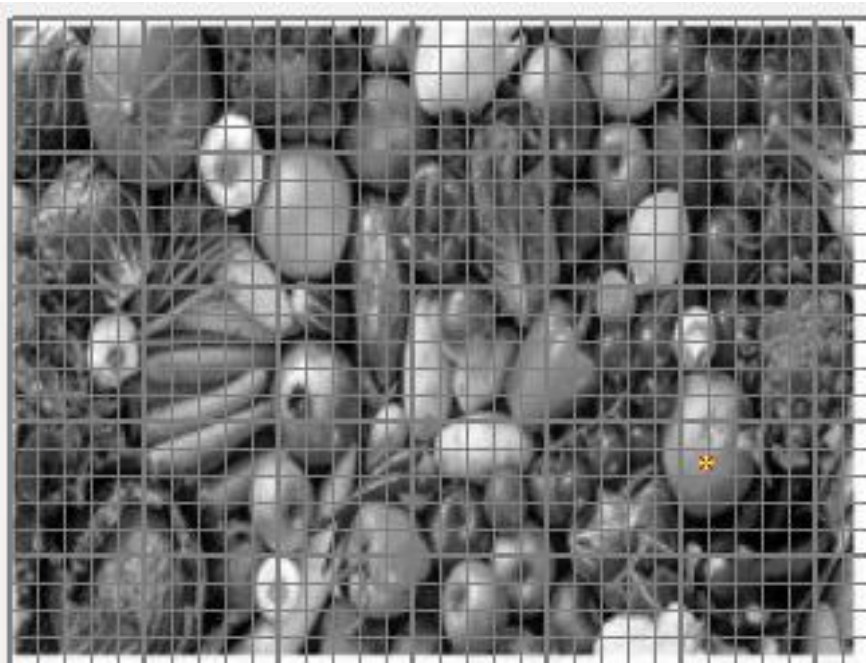
23.4 – fourth important function – the “main” function

```
71 // Main function
72 int main() {
73     unsigned int Hist[PIXEL_VALUES];
74     unsigned int Int_Of_Hist[PIXEL_VALUES];
75
76     // First image
77     LoadGrayImageFromGrayBmpFile(ProccesIMG, "A20gray.bmp");
78     fill_hist_and_calc_int((unsigned char*)ProccesIMG, Hist, Int_Of_Hist, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
79     InitHISTLUT(LUT, Int_Of_Hist);
80     ApplyLUTOnGrayImage((unsigned char*)ProccesIMG, LUT, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
81     StoreGrayImageAsGrayBmpFile(ProccesIMG, "A230.bmp");
82
83     // Second image
84     LoadGrayImageFromGrayBmpFile(ProccesIMG, "A211.bmp");
85     fill_hist_and_calc_int((unsigned char*)ProccesIMG, Hist, Int_Of_Hist, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
86     InitHISTLUT(LUT, Int_Of_Hist);
87     ApplyLUTOnGrayImage((unsigned char*)ProccesIMG, LUT, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
88     StoreGrayImageAsGrayBmpFile(ProccesIMG, "A231.bmp");
89
90     // Third image
91     LoadGrayImageFromGrayBmpFile(ProccesIMG, "A212.bmp");
92     fill_hist_and_calc_int((unsigned char*)ProccesIMG, Hist, Int_Of_Hist, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
93     InitHISTLUT(LUT, Int_Of_Hist);
94     ApplyLUTOnGrayImage((unsigned char*)ProccesIMG, LUT, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
95     StoreGrayImageAsGrayBmpFile(ProccesIMG, "A232.bmp");
96
97     // Fourth image
98     LoadGrayImageFromGrayBmpFile(ProccesIMG, "A221.bmp");
99     fill_hist_and_calc_int((unsigned char*)ProccesIMG, Hist, Int_Of_Hist, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
100    InitHISTLUT(LUT, Int_Of_Hist);
101    ApplyLUTOnGrayImage((unsigned char*)ProccesIMG, LUT, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
102    StoreGrayImageAsGrayBmpFile(ProccesIMG, "A233.bmp");
103
104    // Fifth image
105    LoadGrayImageFromGrayBmpFile(ProccesIMG, "A222.bmp");
106    fill_hist_and_calc_int((unsigned char*)ProccesIMG, Hist, Int_Of_Hist, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
107    InitHISTLUT(LUT, Int_Of_Hist);
108    ApplyLUTOnGrayImage((unsigned char*)ProccesIMG, LUT, NUMBER_OF_ROWS, NUMBER_OF_COLUMNS);
109    StoreGrayImageAsGrayBmpFile(ProccesIMG, "A234.bmp");
```

```
111     WaitForUserPressKey();
112     return 0;
113 }
```

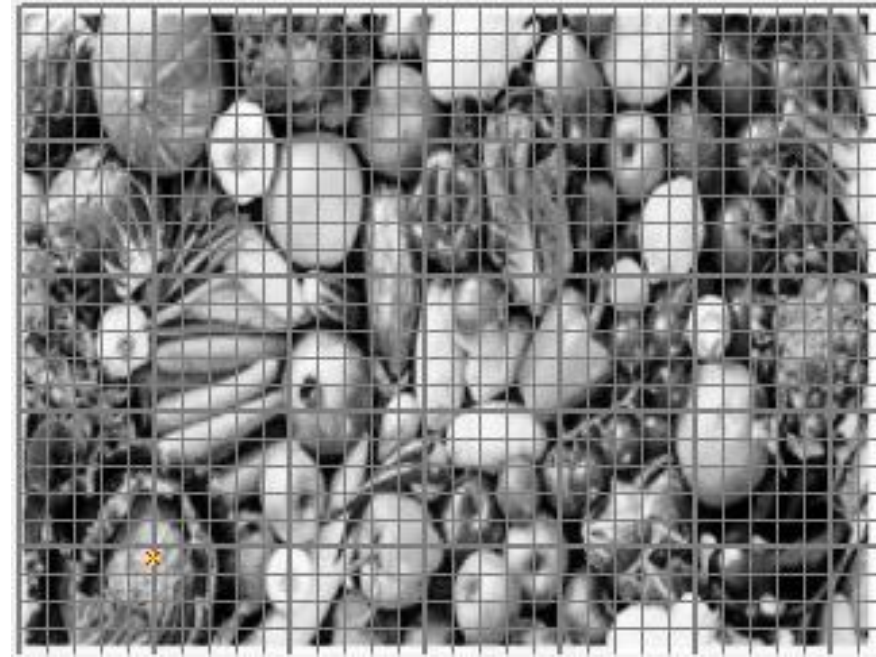
23.5 – demonstration using profiler – **part 1/5**

A20gray.bmp



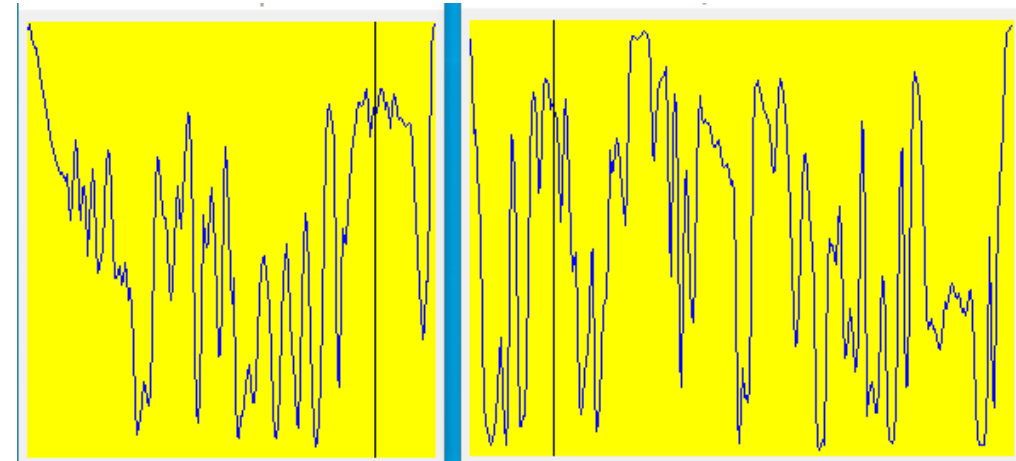
After
process

A230.bmp



Point:	Mouse	Expl.	(M-E)
Row	35	35	0
Column	49	49	0
Red	204	204	0
Green	204	204	0
Blue	204	204	0

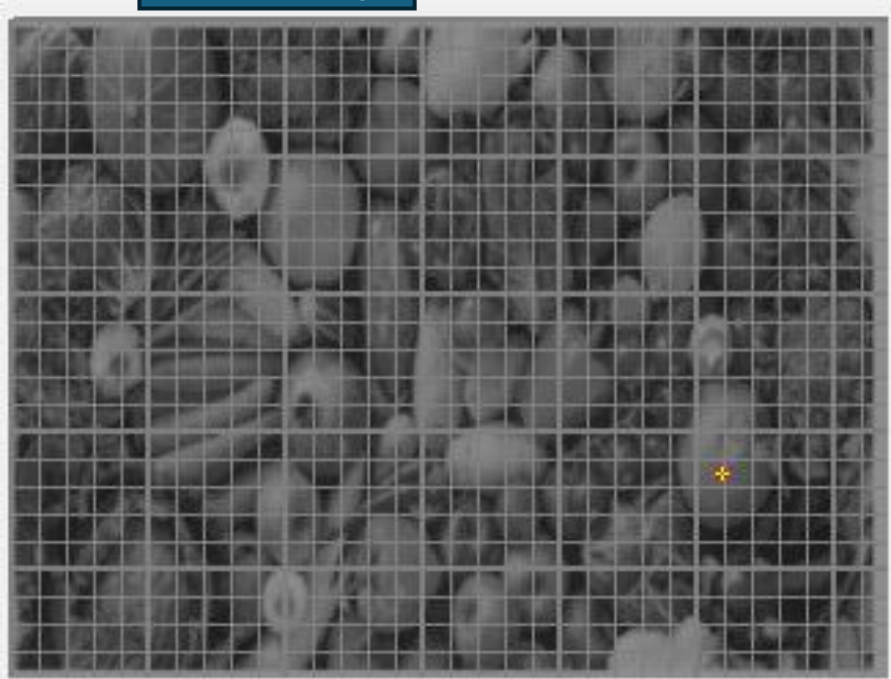
We can see that after activating the "Histogram Equalization" the image changed slightly in that it came out a little darker and sharper.



23.5 – demonstration using profiler – **part 2/5**

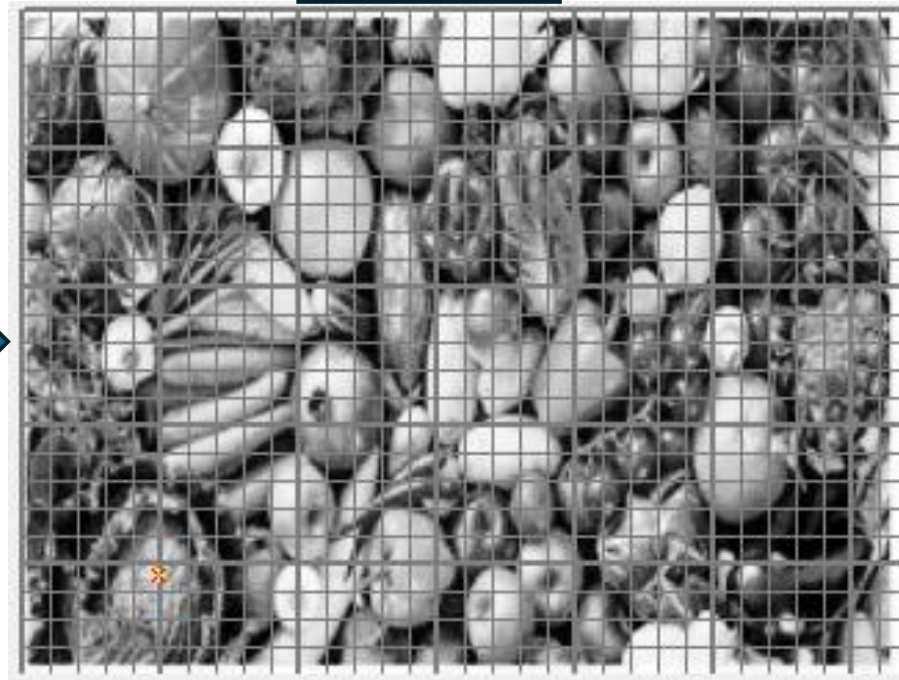
9

A211.bmp



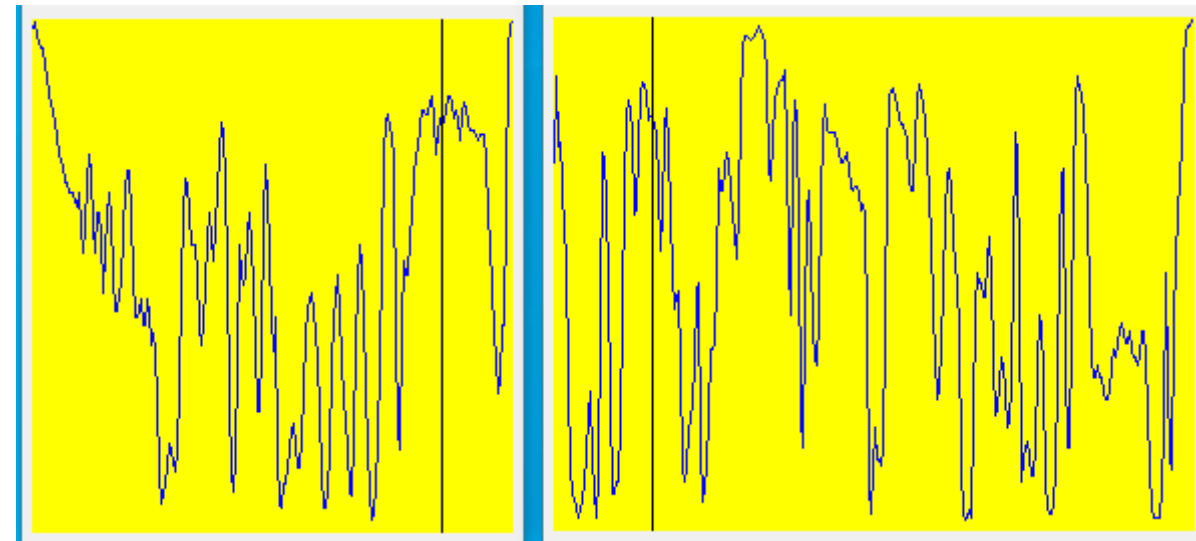
After
process

A231.bmp



Point:	Mouse	Expl.	(M-E)
Row	35	35	0
Column	49	49	0
Red	206	206	0
Green	206	206	0
Blue	206	206	0

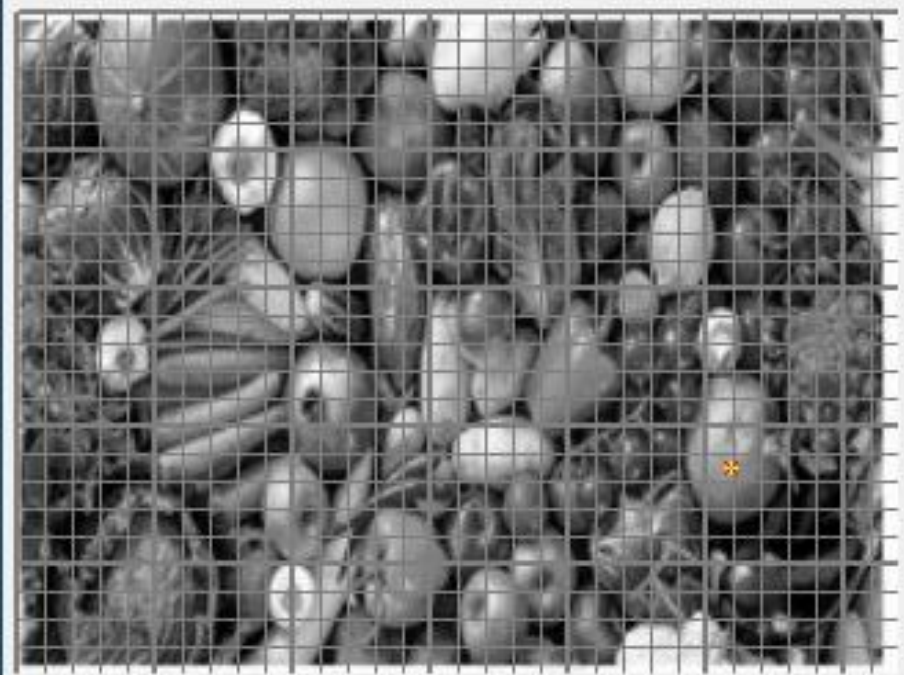
Here we can see that the algorithm worked very well on the image. The ruined image has been corrected very well to the point of clear and sharp colors (relative to the ruined image).



23.5 – demonstration using profiler – **part 3/5**

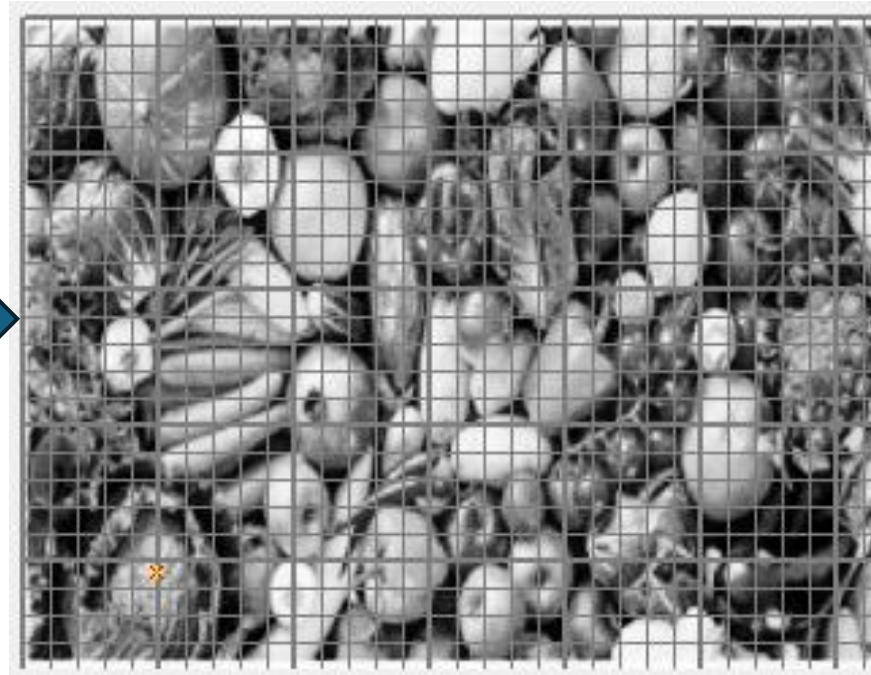
10

A212.bmp



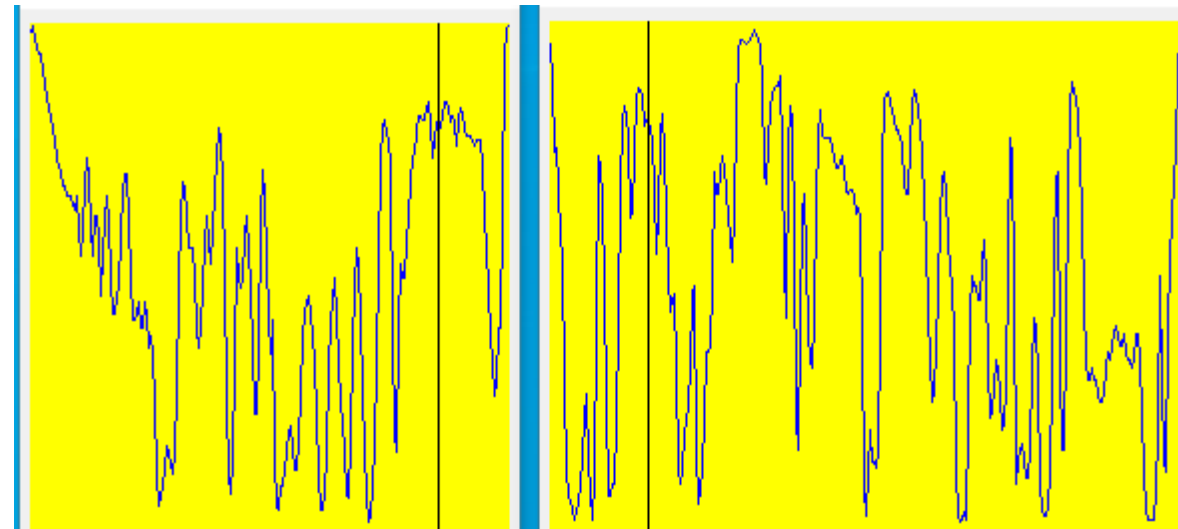
After
process

A231.bmp



Point:	Mouse	Expl.	(M-E)
Row	35	35	0
Column	49	49	0
Red	206	206	0
Green	206	206	0
Blue	206	206	0

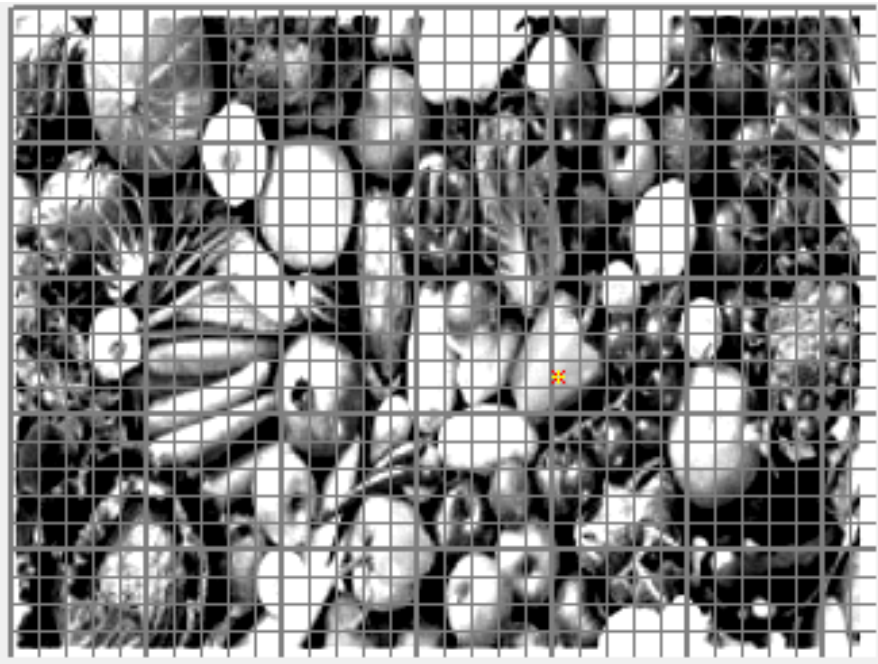
Because the image on which the algorithm was performed is very similar in color to the image “A20gray.bmp” we see that the algorithm worked in a very similar way and there are almost the same results.



23.5 – demonstration using profiler – **part 4/5**

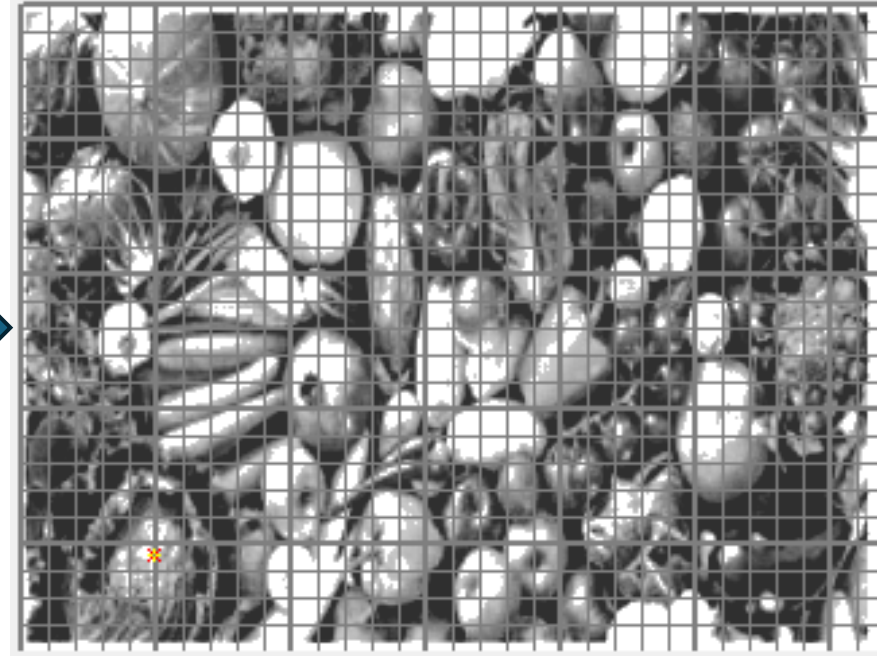
11

A221.bmp



After
process

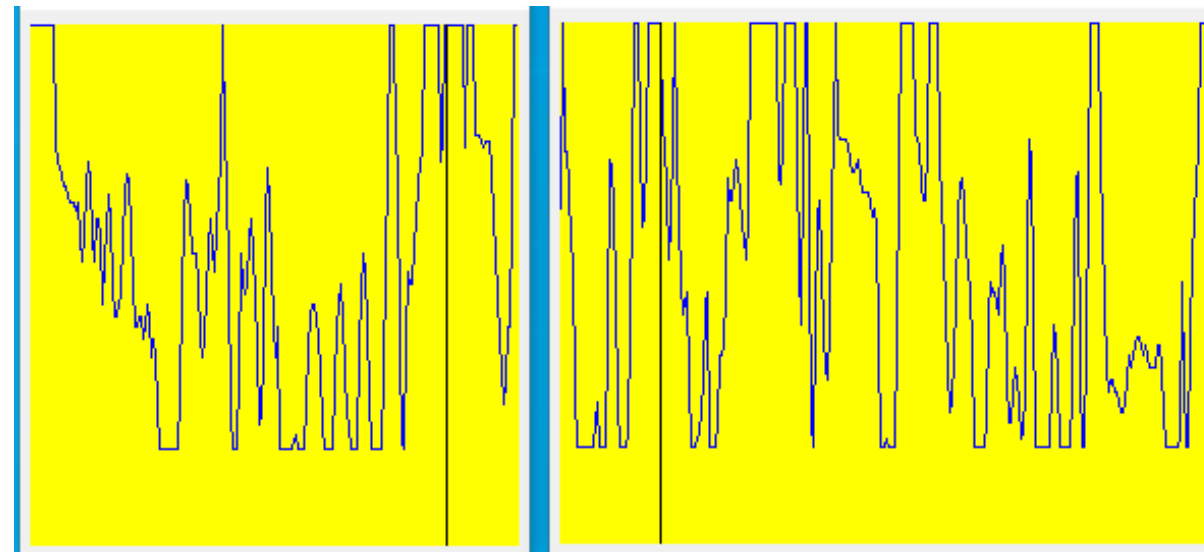
A233.bmp



Point:	Mouse	Expl.	(M-E)
Row	35	35	0
Column	49	49	0
Red	206	206	0
Green	206	206	0
Blue	206	206	0



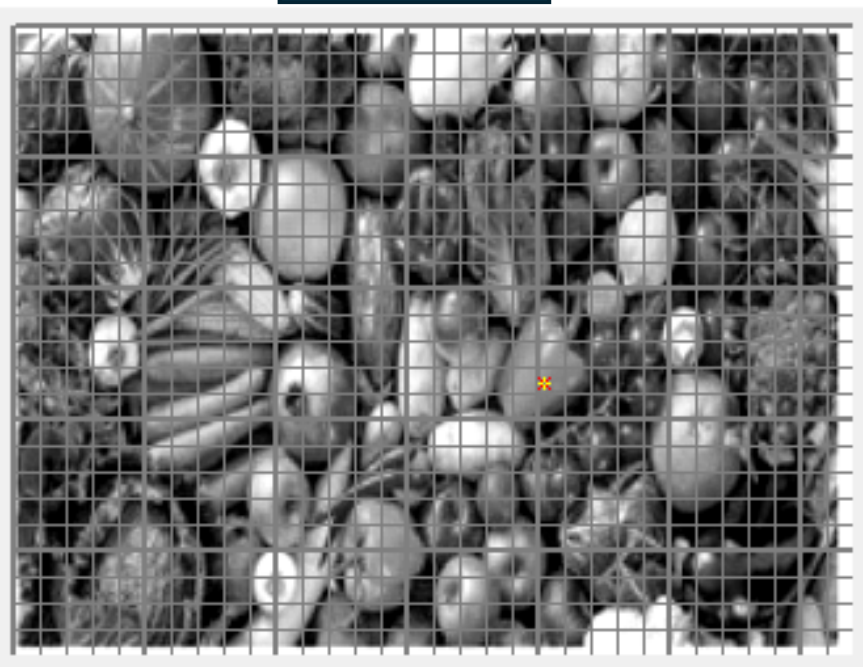
The algorithm failed to correct this image in a good way.
The result is that the image produced becomes darker.



23.5 – demonstration using profiler – **part 5/5**

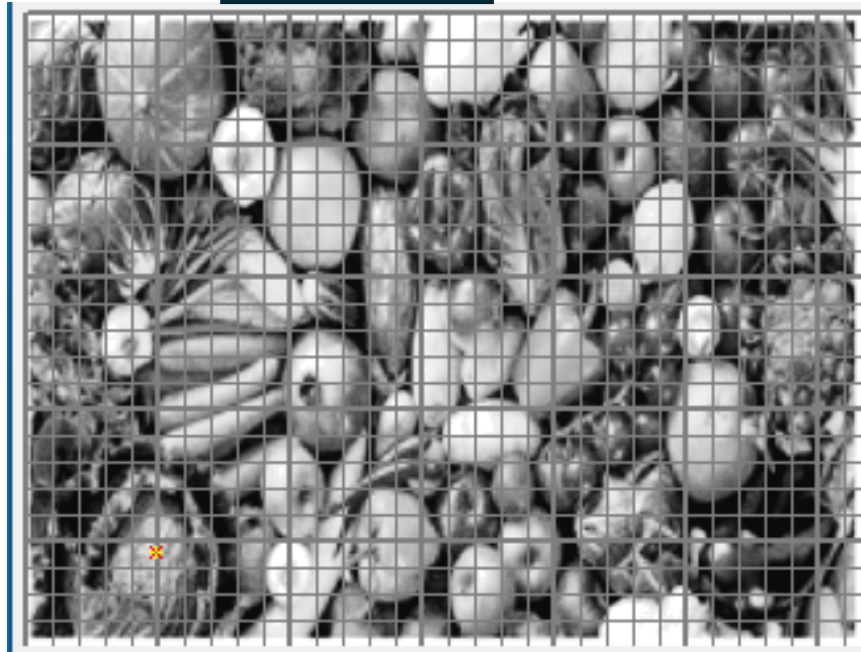
12

A222.bmp



After
process

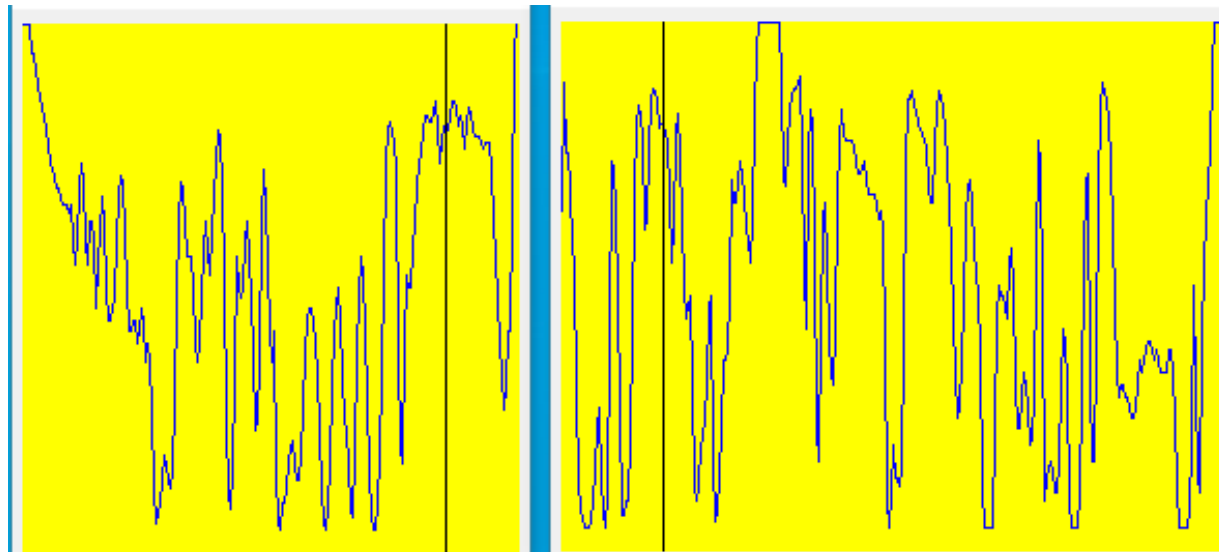
A234.bmp



Point:	Mouse	Expl.	(M-E)
Row	35	35	0
Column	49	49	0
Red	206	206	0
Green	206	206	0
Blue	206	206	0



Here too, the algorithm failed to correct well enough.
The image become lighter and less “authentic”.



23.6 –conclusions

On the ruined image (A211) the algorithm worked best, and an improvement can be seen. On images with too high a contrast the image is slightly destroyed. Images that came out good were slightly destroyed. Overall conclusion: Low-contrast images, although more "ruined" than high-contrast images, the algorithm worked better.

Important Conclusions : (based on theoretical study about histogram equalization process)

1. The best results of the process come from low contrast images – AKA images where the pixels values are concentrated in a narrow range of intensities. These kind of images benefit the most from histogram equalization because it spreads out the most frequent intensity values – thereby improving the global contrast of the image.
2. The poorest results of the process come from already high contrast images –AKA images that have good distribution of pixel values across the intensity range. Applying histogram equalization to these kind of images can introduce unwanted artifacts and noise – making the image look even worse.

23.7 What did we learned?

1. How to use histogram equalization algorithm to improve quality and contrast of given images.
2. We learned that histogram equalization does not always works well – it can even create unwanted artifacts and harm the given image.